

Asset Portfolio Management — A Tutorial Guide

A Frappe/ERPNext app for tracking personal and family investment portfolios: stocks, mutual funds, bonds, fixed deposits, savings accounts, PF/PPF/NPS — with returns tracking (XIRR), consolidated reporting, and optional accounting postings.

1. What This App Does

Asset Portfolio Management is a standalone Frappe app that lets you record every investment a person (or family) holds, keep it updated as its value changes, and see consolidated performance across the whole portfolio — without needing a full accounting/ERP setup.

At its core is one central document, **Asset Portfolio**, which represents a single investment (e.g., "100 shares of Infosys," "HDFC Equity Fund," "5-Year NSC Bond," "SBI Fixed Deposit"). Depending on the **Asset Type** you choose, the form reveals a different set of fields relevant to that instrument, and — for Stocks and Mutual Funds — a companion **Transaction Log** doctype lets you record individual buy/sell trades that automatically roll up into the parent asset.

Supported asset types:

Asset Type	Tracked Via
Stock	Stock Transaction Log (buy/sell)
Mutual Fund	Mutual Fund Transaction Log (buy/sell)
Bond	Bond Interest Log + Bond Principal Log (repayment schedules)
Fixed Deposit	FD Interest Log + FD Principal Log (repayment schedules)
Savings Account	Savings Account Transaction Log (deposits/withdrawals)
PF / PPF / NPS	Tracked generically via Investment/Current Value fields

2. Core Concepts

2.1 Portfolio Owner

Before creating any asset, create a **Portfolio Owner** — the person the investment belongs to (useful when tracking investments for multiple family members under one system). It just needs an **Owner Name** and **Email**.

Known issue: the `_owner` field on Asset Portfolio collides with Frappe's built-in `_owner` metadata field (the document creator). In the current build this can prevent the field from saving correctly through the standard UI save flow in some cases. If you hit a "Could not find Owner" validation error, this is why — it's on the list of things to fix.

2.2 Asset Portfolio — The Central Record

Every investment starts here. Common fields across all types:

- **Asset ID** — unique identifier (used as the document name)
- **Asset Name, Asset Type, Provider/Institution**
- **Owner** (Portfolio Owner), **Status** (Active / Matured / Closed)
- **Date of Investment, Maturity Date** (for Bonds/FDs)
- **Investment Value, Current Value, XIRR, Total Gain/Loss, Total Return %**

Below these, a **type-specific section** appears based on `Asset Type` (Stock Details, Mutual Fund Details, Bond Details, Fixed Deposit Details, Savings Account Details).

2.3 Transaction Logs (Stock & Mutual Fund)

For Stocks and Mutual Funds, you don't manually edit "Quantity Held" or "Units Held" — instead, you **submit transaction documents**, and the app recalculates the parent asset automatically:

- **Stock Transaction Log** — Buy/Sell with Quantity, Price, Brokerage, Taxes & Fees
- **Mutual Fund Transaction Log** — Buy/Sell with Units, NAV, Stamp Duty (buy) / Exit Load (sell)

Both use a **weighted-average-cost** engine: every Buy recalculates the running average cost; every Sell is measured against that average to compute a realized gain or loss — this mirrors how mutual fund and stock cost-basis accounting works in practice.

2.4 XIRR (Extended Internal Rate of Return)

Every Asset Portfolio record shows an **XIRR** — the annualized return accounting for the exact timing of cash flows (not just start/end value). It's calculated automatically via `calculate_asset_xirr()` using a Newton-Raphson solver, built from:

- Stock: all Buy/Sell transaction amounts + current value (if still Active)
 - Mutual Fund: same pattern via Mutual Fund Transaction Log
 - Bond/FD: interest and principal repayment schedule entries + remaining value
 - Savings Account: deposits/withdrawals + current balance
 - PF/PPF/NPS: a simple point-to-point estimate using current value
-

3. Tutorial: Tracking a Stock Investment

1. **Create a Demat Account** (Setup once) — e.g., "Zerodha - XXXX1234".
2. **Create an Asset Portfolio** record: - Asset Type: `Stock` - Fill Asset Name, Owner, Provider, Date of Investment - Under **Stock Details**: link the Demat Account, set Industry/Stock Category, and set **Current Market Price** (you'll update this periodically as the market price changes)
3. **Record a Buy** — create a **Stock Transaction Log**: - Asset: your Asset Portfolio record - Transaction Type: `Buy`, Quantity, Price, Brokerage, Taxes & Fees - **Submit** it -> the parent Asset Portfolio's Quantity Held, Average Purchase Price, and Investment Value update automatically.
4. **Record more Buys** as you accumulate more shares — the average purchase price re-weights each time.
5. **Record a Sell** — create another Stock Transaction Log with Transaction Type `Sell`: - On submit, the app computes **Realized Gain/Loss** = Sale proceeds - (quantity sold × average cost at the time), and books it directly on that transaction record. - The parent asset's Quantity Held drops accordingly; its Average Purchase Price is unaffected by sells (only Buys change the average).
6. Update **Current Market Price** on the Asset Portfolio periodically to keep unrealized Total Gain/Loss and XIRR current.

4. Tutorial: Tracking a Mutual Fund Investment (Buy -> Hold -> Sell)

This is the newest addition to the app and follows the same weighted-average-cost pattern as Stocks, with NAV in place of price and Units in place of quantity.

Step 1 — Create the Asset

Create an **Asset Portfolio** with: - Asset Type: `Mutual Fund` - Under **Mutual Fund Details**: Folio Number, AMC Name, Scheme Category (Equity/Debt/Hybrid/Index/ELSS/Liquid), and **Current NAV**

Step 2 — Buy Units

Create a **Mutual Fund Transaction Log**: - Transaction Type: `Buy` - Units, NAV, Stamp Duty - **Submit** -> Total Amount = (Units × NAV) + Stamp Duty. The parent asset's Units Held and Average NAV (Cost) update.

Buy again at a different NAV — the average cost NAV re-weights across both purchases, exactly like the stock example.

Step 3 — Hold

While holding, periodically update **Current NAV** on the Asset Portfolio. The app recalculates: - `Current Value = Units Held × Current NAV` - `Total Gain/Loss = Current Value - Investment Value` (unrealized gain/loss on what you're still holding)

Step 4 — Sell Units (Profit or Loss)

Create another **Mutual Fund Transaction Log** with Transaction Type **Sell**, the Units being sold, the NAV at sale, and any Exit Load:

On submit, the transaction records: - **Average Cost NAV at Transaction** — the weighted-average cost basis at the moment of sale - **Gain / Loss per NAV (Unit)** = Sale NAV - Average Cost NAV — booked on every sell transaction so you can see the per-unit result of each individual trade, not just the aggregate - **Realized Gain / Loss** = (Gain/Loss per Unit × Units Sold) - Exit Load

This means both the *aggregate* realized P&L and the *per-NAV* (per-unit) P&L are permanently recorded on the transaction for audit/tracking purposes — exactly matching how a fund statement shows realized gain per redemption.

Example (verified in testing):

Step	Units	NAV	Result
Buy 1	100	100	Avg cost NAV -> 100.0
Buy 2	100	110	Avg cost NAV -> 105.1 (incl. stamp duty)
Sell 1	80	130	Gain/unit +24.9, Realized Gain INR 1,987
Sell 2	50	90	Loss/unit -15.1, Realized Loss -INR 760
Remaining	70	—	Avg cost NAV unchanged at 105.1

Step 5 (Optional) — Post to the Books

See [Section 7 — Accounting Integration](#) below.

5. Tutorial: Tracking Bonds & Fixed Deposits

Bonds and FDs don't use a transaction log — instead they use **repayment schedule child tables** on the Asset Portfolio itself:

1. Create the Asset Portfolio with Asset Type **Bond** or **Fixed Deposit**, fill in face value / interest rate / maturity date, etc.
2. Add rows to the **Bond Interest Schedule** / **FD Interest Schedule** table as interest becomes due, marking each row **Paid** or **Pending** once received.
3. Add rows to the **Bond Principal Schedule** / **FD Principal Schedule** table as principal is repaid (e.g., at maturity, or in installments).

4. The app recalculates on save: - Interest Paid / Interest Unpaid from the schedule -
 $\text{Principal Unpaid} = \text{Investment Value} - \text{Principal Paid} - \text{Current Value} = \text{Principal Unpaid} - \text{Total Gain/Loss} = (\text{Current Value} + \text{Interest Paid} + \text{Principal Paid}) - \text{Investment Value}$

6. Tutorial: Tracking a Savings Account

1. Create an Asset Portfolio with Asset Type `Savings Account` and an Account Number.
2. Record each **Savings Account Transaction Log** entry as a Deposit or Withdrawal.
3. XIRR treats deposits as outflows and withdrawals (plus the current balance) as inflows.

7. Accounting Integration (Mutual Funds)

By default, this app is a **pure tracking layer** — it does not touch the Chart of Accounts. Optionally, for Mutual Fund transactions, you can enable **automatic draft Journal Entry creation** so every buy/sell shows up in your books for review.

Setup (once per Mutual Fund asset)

On the Asset Portfolio record, open the **Mutual Fund Accounting** section and set: - **Company - Investment Account** (an asset-side ledger, e.g. "Short-term Investments") - **Settlement Account** (the bank/cash account funds move through) - **Realized Gain / Loss Account** (only needed once you start selling)

If any of these are left blank, transactions still post normally — the app just skips Journal Entry creation and shows an alert telling you what to configure.

What Gets Posted

On Buy: | Account | Debit | Credit | |---|---|---| | Investment Account | Total Amount (incl. stamp duty) | | | Settlement Account | | Total Amount |

On Sell: | Account | Debit | Credit | |---|---|---| | Settlement Account | Net proceeds (after exit load) | | | Investment Account | | Cost basis of units sold | | Gain/Loss Account | (*if loss*) | (*if gain*) |

The Journal Entry is created in **Draft** status (`docstatus = 0`) — nothing hits the general ledger until someone in Accounts reviews and submits it manually. The transaction log stores a link back to it in its **Journal Entry (Draft)** field.

Cancelling a transaction automatically deletes its still-draft Journal Entry. If the Journal Entry was already submitted by someone in Accounts before the cancellation, it's left untouched with a warning instead (so nothing is silently reversed in the ledger).

This accounting hook currently exists for **Mutual Fund Transaction Log** only. Stock Transaction Log can be extended the same way if needed.

8. Reports & Dashboard

8.1 Portfolio Dashboard (Workspace Page)

A visual overview page showing: - **KPI cards**: Total Investment, Current Value, Total Gain/Loss, Overall Return % - **Charts**: Asset Allocation by Type, and by Provider

Backed by the whitelisted API method `get_dashboard_data()` , which aggregates every Asset Portfolio record by type, provider, and owner.

8.2 Consolidated Portfolio Report

A filterable table of every asset (by Owner, Asset Type, Status) showing Investment Value, Current Value, Gain/Loss, Return %, and XIRR side by side, with a summary panel showing portfolio-wide totals and weighted average return.

8.3 Portfolio Allocation Report

Groups your holdings by a dimension of your choice — Asset Type, Provider, Owner, Industry, Stock Category, or Bond Rating — and shows a percentage allocation pie/bar chart plus a breakdown table.

8.4 Upcoming Maturity Report

Lists all **Active** Bonds and Fixed Deposits with a maturity date, sorted soonest-first, with "Days Remaining" — useful for planning reinvestment or renewal decisions. Supports a "days horizon" filter (e.g., show only what matures in the next 90 days).

9. Quick Reference — Doctypes in This App

Doctype	Purpose
Portfolio Owner	The person an asset belongs to
Asset Portfolio	The central investment record (all types)
Demat Account	Brokerage account used by Stock/Bond holdings
Stock Transaction Log	Buy/Sell log for Stocks
Mutual Fund Transaction Log	Buy/Sell log for Mutual Funds, with NAV gain/loss booking and optional Journal Entry creation
Bond Interest Log / Bond Principal Log	Repayment schedule rows for Bonds

Doctype	Purpose
FD Interest Log / FD Principal Log	Repayment schedule rows for Fixed Deposits
Savings Account Transaction Log	Deposit/Withdrawal log for Savings Accounts

10. Tips & Known Limitations

- **Update market prices/NAVs regularly** — unrealized Gain/Loss, Total Return %, and XIRR are only as current as the price/NAV fields you maintain.
- **Never edit Quantity Held / Units Held / Average Cost directly** for Stocks or Mutual Funds — always go through a Transaction Log so the weighted-average-cost math stays correct and traceable.
- **Submitted transactions are the source of truth** — draft (unsubmitted) transactions are ignored by all rollups and reports.
- **The `owner` field collision** (see Section 2.1) is a known pre-existing bug affecting every asset type's Owner field, not specific to any one feature.
- **Accounting postings are currently Mutual-Fund-only** and always land as **drafts** — they're a starting point for your books, not an auto-posting engine.